



Este documento é um guia da atividade desenvolvida em conjunto no laboratório de AEDs II. Nesta atividade incluiremos duas funcionalidades no nosso sistema de comércio de produtos. A atividade é individual e seu conteúdo servirá como base para atividades avaliativas futuras. Portanto, atenção ao que será desenvolvido em aula e procure fazer sua parte na tarefa, enviando as atualizações para o repositório da atividade no GitHub.

Tema: Relatório dos pedidos de um produto

Temos nosso sistema de comércio de produtos armazenando seus dados em estruturas de árvore balanceada. Algumas questões de desempenho e usabilidade ainda permanecem:

- *Podemos criar mais de uma árvore se quisermos ter diversas possibilidades para as operações de busca. Nestes casos, será que precisamos recarregar todos os dados do arquivo para cada árvore?*
- *Além disso, queremos uma resposta eficiente para uma pergunta comum: “quais pedidos contêm um produto escolhido pelo usuário”? Percorrer todos os pedidos e, dentro deles, toda a lista de produtos não é uma operação com custo baixo.*
- *Finalmente, outra melhoria seria enviar os resultados de relatórios muito grandes para arquivos, em lugar de mostrar os resultados na tela.*

Vamos tentar melhorar estes aspectos e adaptar o programa principal para que funcionem.

Pré-Atividade: Comparando ABB e AVL na prática

Na última aula vocês implementaram as classes ABB e AVL. Antes de seguirmos com a oficina, vamos observar experimentalmente o impacto do balanceamento sobre o desempenho de busca.

Crie uma classe de teste (por exemplo, TestePreAtividade) com um método main que:

1. Gere uma lista com $N = 10.000$ valores inteiros aleatórios, usando Random com semente fixa (new Random(42)), para que o experimento seja reproduzível. I
2. Insira essa mesma sequência de valores, na mesma ordem, em uma ABB<Integer, Integer> e em uma AVL<Integer, Integer> (use o próprio valor como chave e como item).
3. Gere uma segunda lista com $M = 1.000$ valores aleatórios, representando "pedidos de busca" (alguns estarão nas árvores, outros não).
4. Para cada valor dessa segunda lista, pesquise-o nas duas árvores e acumule o total de comparações (getComparacoes()) e o tempo total (getTempo()) gasto por cada estrutura.
5. Repita os passos 2 a 4, mas agora inserindo os N valores em ordem crescente nas duas árvores.

Organize os resultados em uma tabela com quatro linhas (ABB/inserção aleatória, AVL/inserção aleatória, ABB/inserção ordenada, AVL/inserção ordenada) e duas colunas (total de comparações, tempo total).

Com base na tabela, responda:

1. Em qual cenário a ABB tem desempenho parecido com o da AVL? Por quê?
2. Em qual cenário a diferença entre ABB e AVL é mais evidente? O que acontece com a estrutura da ABB nesse caso?
3. Considerando que a ordem dos produtos no arquivo produtos.txt não segue nenhum critério garantido, o que isso justifica sobre a escolha de AVL para produtosPorId no aplicativo principal?

Tarefa 1: (em aula)

Seguindo as instruções e acompanhando a explicação dos professores, vamos carregar a árvore de Produtos por nome a partir de uma árvore já existente.

Tarefa 2: (em aula)

Seguindo as instruções e acompanhando a explicação dos professores, observe o funcionamento do gerador de Pedidos aleatórios. Queremos que estes pedidos sejam registrados juntos aos produtos que eles contêm. Como a classe Produto não tem uma lista de Pedidos, observe a solução proposta com tabela *hash*.

Implemente o seguinte método na classe App:

```
private static void inserirNaTabela(Produto produto, Pedido pedido)
```

Este método deve incluir corretamente um Pedido na tabela *hash* que faz a separação dos pedidos pelos produtos.

Tarefa 3: (em aula)

O relatório de pedidos de um produto é muito longo para ser exibido na tela. No aplicativo, implemente o método

```
static void pedidosDoProduto()
```

Este método deve gerar, em arquivo, o relatório de todos os pedidos de um produto escolhido pelo usuário.

Tarefa 4: (para casa)

Revise todo o código para se preparar para a atividade pontuada da semana que vem. Identifique os pontos sem robustez no aplicativo (isto é, problemas causados por exceções na operação incorreta da estruturas de dados) e faça os tratamentos adequados.

Outras observações:

- O projeto deve estar hospedado na tarefa correspondente do GitHub Classroom. Endereço para aceitar a tarefa: **mesmo da oficina Árvores Binárias de Busca**
- As atividades pontuadas da disciplina podem depender direta ou indiretamente dos códigos desenvolvidos nas aulas, portanto, é essencial o comprometimento no acompanhamento das atividades semanais.
- Para a correção das atividades pontuadas, serão considerados todos os *commits/pushes* realizados ao longo das semanas, não somente o último com a resposta final do exercício.